Instructions:

All assignment programs should first be written by hand in a notebook, and then implemented and executed in Jupyter Notebook.

Please include proper comments in the code to make it easy to understand and readable.

Question 1.

Complete the following steps:

- Create a class called `Person`
- Add an `__init__` method that takes `name` and `age` as parameters
- Add a method called `greet` that prints **"Hello, my name is "** followed by the name
- Create an object `p1` of the class with name **"John"** and age **36**
- Call the `greet` method on `p1`

Solution: [Classes/Objects]

```
# Create a class

class Person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def greet(self):
    print("Hello, my name is " + self.name)

# Create an object
p1 = Person("John", 36)

# Call the greet method
p1.greet()
```

Question 2.

Complete the following steps:

- Create a class called `Dog`
- Add an `__init__` method that takes `name` and `age`, and store them as properties using `self`

- Add a method called `bark` that prints the dog's name followed by **" says Woof!"**
- Create an object `d1` of the `Dog` class with name **"Buddy"** and age **3**
- Call the `bark` method on `d1`

Solution: [__init__ Method]

```
# Create the Dog class
class Dog:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def bark(self):
    print(self.name + " says Woof!")

# Create an object
d1 = Dog("Buddy", 3)

# Call the bark method
d1.bark()
```

Question 3.

Complete the following steps:

- Create a class called `Car`
- Add an `__init__` method with a `brand` parameter, and store it as a property
- Add a method called `show` that prints the brand
- Create an object `c1` of the `Car` class with with brand **"Ford"**
- Call the `show` method on `c1`

Solution: [self Parameter]

```
# Create the Car class
class Car:
  def __init__(self, brand):
    self.brand = brand

  def show(self):
    print(self.brand)

# Create an object
c1 = Car("Ford")

# Call the show method
c1.show()
```

Question 4.

Complete the following steps:

- Create a class `Student` with an `__init__` that takes `name` and `grade`, and stores them as properties
- Create an object `s1` with name **"Anna"** and grade **"A"**
- Print the grade of `s1`
- Change the grade of `s1` to **"B"**
- Print the updated grade

Solution: [Class Properties]

```
# Create the Student class
class Student:
  def __init__(self, name, grade):
    self.name = name
    self.grade = grade

# Create an object
s1 = Student("Anna", "A")

# Print the grade
print(s1.grade)

# Change the grade
s1.grade = "B"

# Print the updated grade
print(s1.grade)
```

Question 5.

Complete the following steps:

- Create a class called `Rectangle`
- Add an `__init__` method with `width` and `height`, and store them as properties
- Add a method called `area` that returns the width multiplied by the height
- Create an object `r1` with width **5** and height **3**
- Print the area of `r1`

Solution: [Class Methods]

```
# Create the Rectangle class
class Rectangle:
  def __init__(self, width, height):
    self.width = width
```

```
    self.height = height

  def area(self):
    return self.width * self.height

# Create an object
r1 = Rectangle(5, 3)

# Print the area
print(r1.area())
```

Question 6.

Complete the following steps:

- Create a parent class `Animal` with an `__init__` that takes `name`
- Add a method `speak` that prints the name
- Create a child class `Dog` that inherits from `Animal`
- Create an object `d1 = Dog("Rex")`
- Call `d1.speak()`

Solution: [Inheritance]

```
# Create the Animal class
class Animal:
  def __init__(self, name):
    self.name = name

  def speak(self):
    print(self.name)

# Create the Dog class (inherits from Animal)
class Dog(Animal):
  pass

# Create an object
d1 = Dog("Rex")

# Call the speak method
d1.speak()
```

Question 7.

Complete the following steps:

- Create a class `Cat` with a method `sound` that prints **"Meow"**
- Create a class `Fox` with a method `sound` that prints **"Wa-pa-pa-pa-pa-pow!"**

- Create objects `c1 = Cat()` and `f1 = Fox()`
- Call `sound()` on both objects using tuple

Solution: [Polymorphism]

```
# Create the Cat class
class Cat:
  def sound(self):
    print("Meow")

# Create the Fox class
class Fox:
  def sound(self):
    print("Wa-pa-pa-pa-pa-pow!")

# Create objects and loop
c1 = Cat()
f1 = Fox()

for animal in (c1, f1):
  animal.sound()
```

Question 8.

Complete the following steps:

- Create a class `ScoreBoard`
- Add an `__init__` with a `score` parameter and store it as a private attribute
- Add a method called `get_score` that returns the private score
- Create an object `s1` with a score of **0**
- Print the score of `s1`

Solution:[Encapsulation]

```
# Create the ScoreBoard class
class ScoreBoard:
  def __init__(self, score):
    self.__score = score

  def get_score(self):
    return self.__score

# Create an object
s1 = ScoreBoard(0)

# Print the score
print(s1.get_score())
```